AI AGENTS

WORKSHOP SERIES

Lora Yovcheva-Clavijo

UCLA MQE Spring 2026

UCLA

SCAN THE QR CODE TO SIGN IN

Access to Materials: https://github.com/lorayc/ai-agents

THE WORKSHOP SERIES

01 **Foundations**

API calls, tool use, ReAct loop, FRED agent

02 **RAG & Memory**

Embeddings, vector stores, FOMC document search

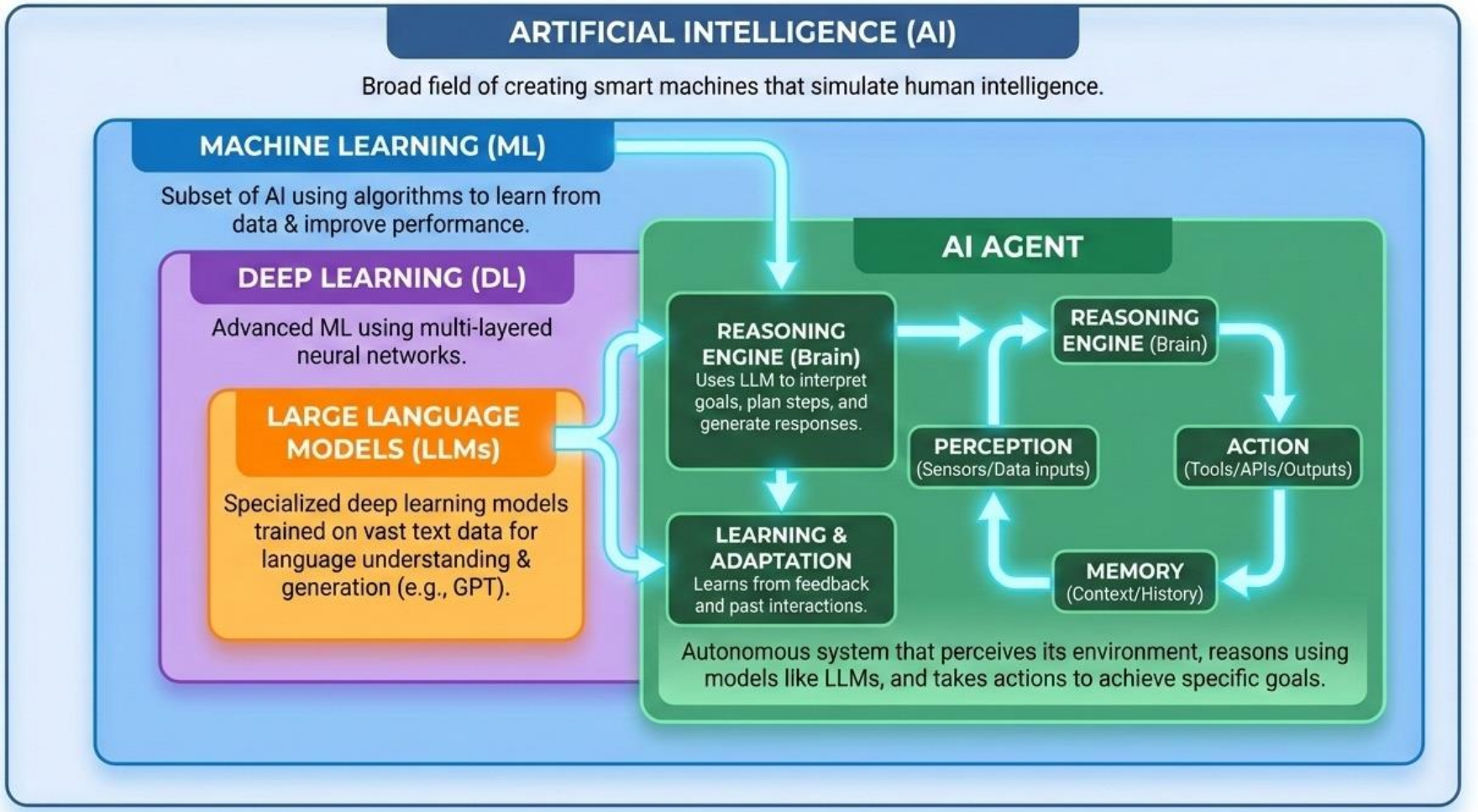
03 **Multi-Agent Systems**

Orchestration, delegation, parallel research

04 **Production & Eval**

Guardrails, evaluation, deployment patterns

THE ECOSYSTEM



HOW GENERATIVE AI WORKS

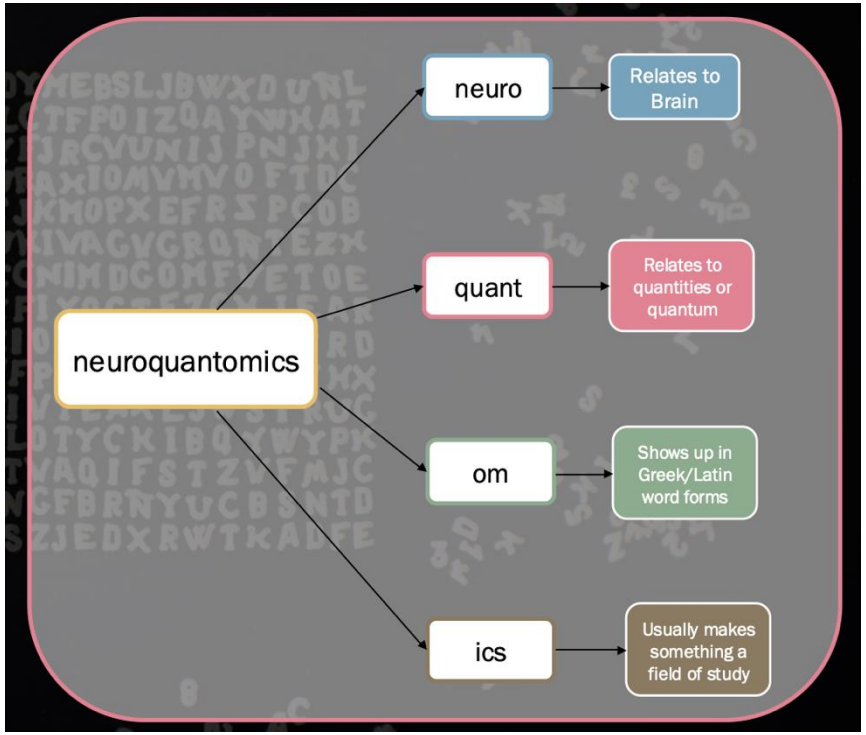
Large Language Models (LLMs)

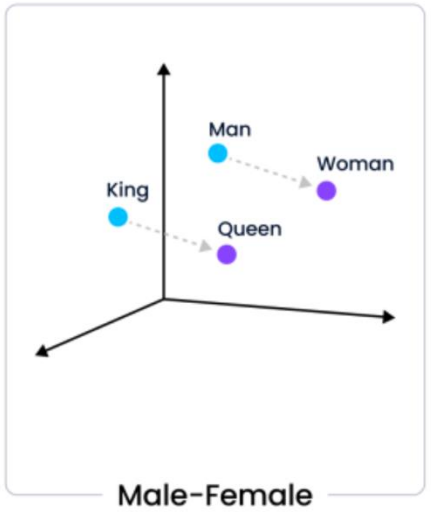
- Trained on massive text corpora to predict the next token
- Encode statistical patterns of language, facts, and reasoning
- Generate responses by sampling from probability distributions
- Context window: all input + output tokens in a single conversation
- No persistent memory between API calls — stateless by default

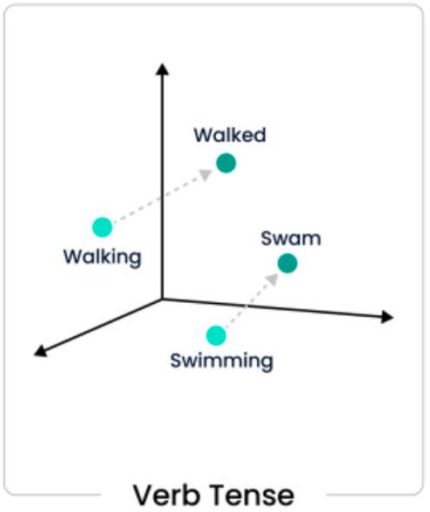
Key Properties

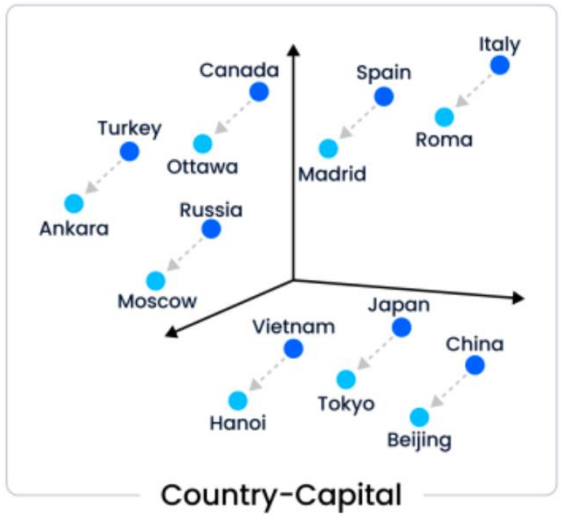
Stochastic	Same input can yield different outputs
Knowledge cutoff	Training data has a fixed end date
No live data	Cannot access the internet by default
Hallucination risk	May confidently produce wrong facts
Tool-augmentable	Can be extended with external functions

VECTOR EMBEDDINGS









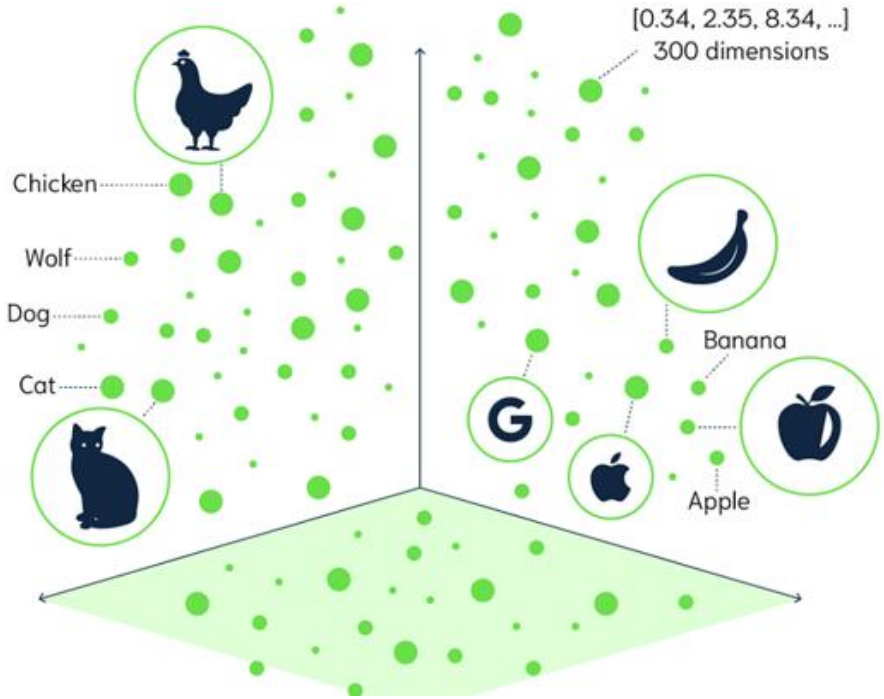
Tokenization breaks raw text into tokens.

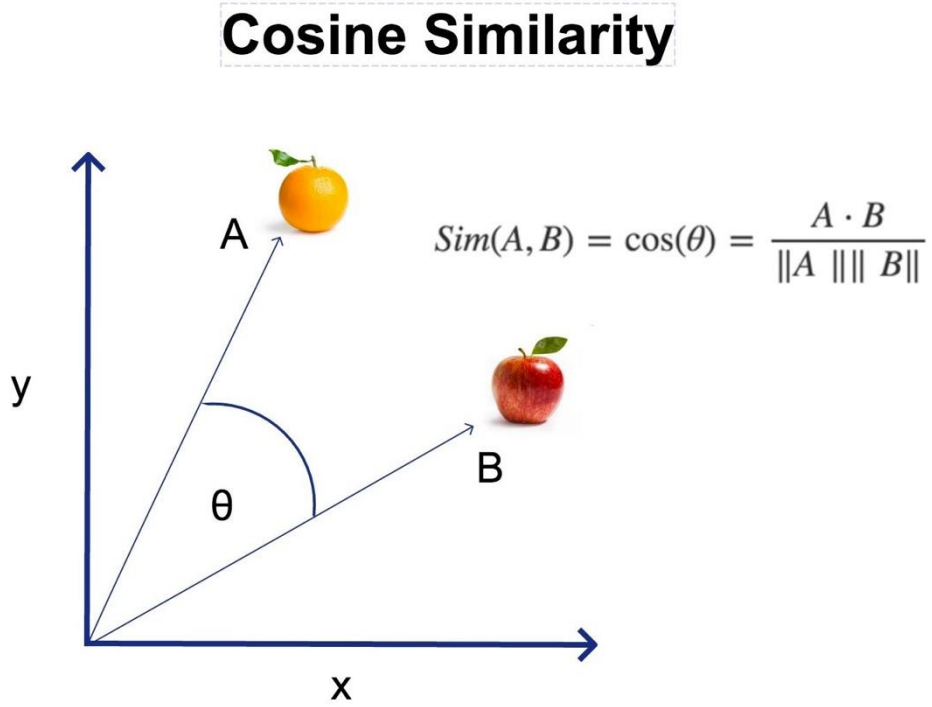
Tokens are converted to token ID. Tokens have no meaning.

Embeddings are vectors that have DIRECTION and MAGNITUDE.

https://platform.openai.com/tokenizer

SEMANTIC MEANING & SIMILARITY SEARCH





Semantic meaning is derived from patterns that the model learns when being trained.

Similarity Search find the vectors that are closest to each other.

vector = embedding = embedded semantic meanings

CHATBOT vs. AGENT

CHATBOT

Stateless & Reactive

`User message → LLM → Response`

- Answers from training data only
- No access to live information
- Single-turn: no planning or iteration
- Numbers may be outdated or wrong

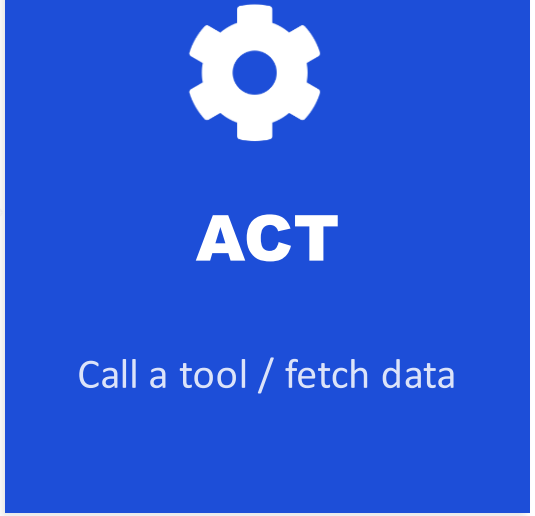
AGENT

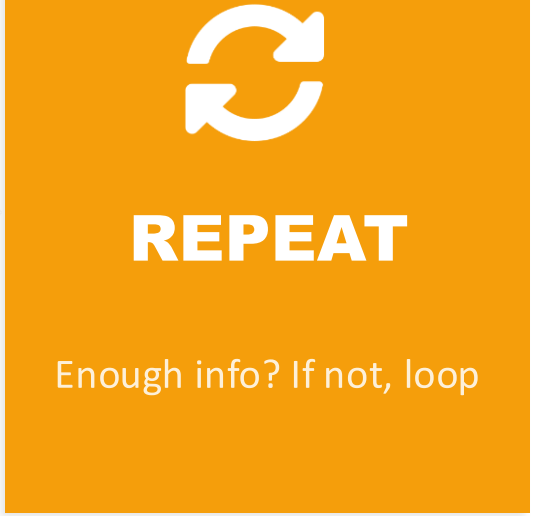
Goal-Directed & Autonomous

`Goal → Reason → Act → Observe → Repeat`

- Calls tools to fetch live data
- Multi-step reasoning and planning
- Decides when it has enough info
- **Grounded in real, current data**

THE ReAct PATTERN





RETURN — Goal met → deliver grounded, data-backed answer to the user

WHY AGENTS FOR FINANCE & ECONOMICS?

"Has the Phillips curve relationship strengthened since 2020?"

1. Decides it needs UNRATE and CPIAUCSL from FRED
2. Pulls both series via tool calls
3. Examines correlation structure across time periods
4. Compares pre- and post-2020 patterns
5. Delivers a data-grounded analysis with real numbers

Live Data Access

Real-time FRED, BLS, Census data — not stale training data

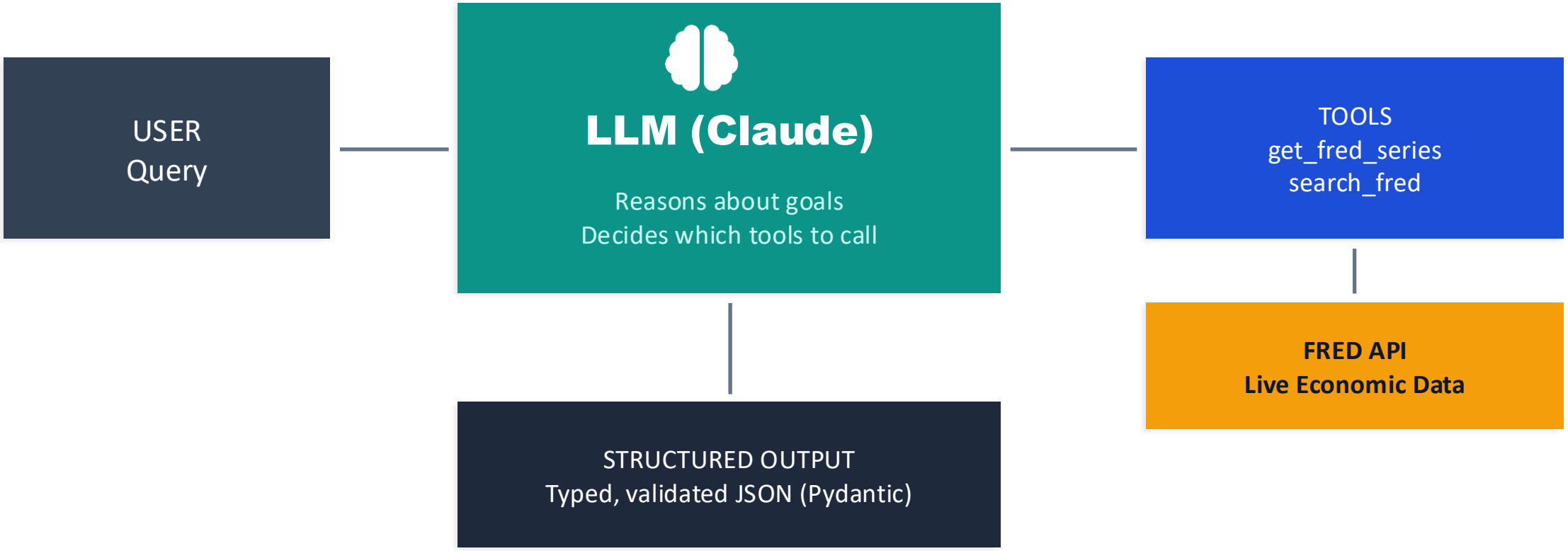
Autonomous Research

Multi-step reasoning: search → retrieve → analyze → synthesize

Reproducible Analysis

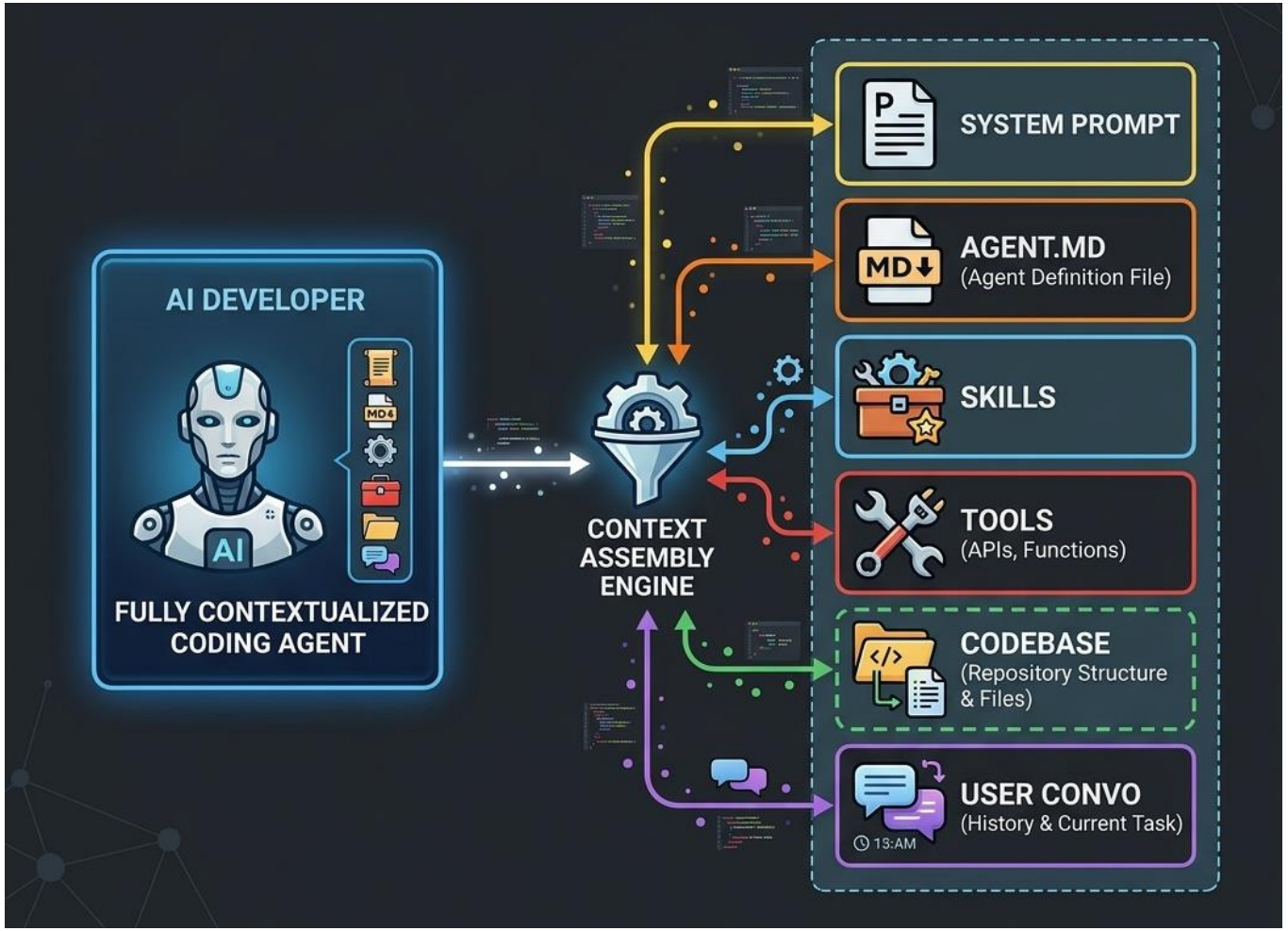
Structured outputs, typed data, programmatic pipelines

AGENT ARCHITECTURE



The loop continues until the LLM decides it has enough information to answer (stop_reason = 'end_turn')

CONTEXT



- Context is the model assembling information in order to execute and act.
- Start with an amount of tokens from this
- Tokens grow as conversation grows.
- E.g.: Claude compacts after 250,000

CREATING AN AI AGENT

NO-CODE / LOW-CODE

- Claude.ai Projects
- OpenAI GPTs
- Zapier / Make / n8n
- Relevance AI, Stack AI, Flowise

IN VS CODE

- Claude Code
- GitHub Copilot Agent Mode
- Continue.dev
- Cursor / Windsurf

PYTHON FRAMEWORKS

- Anthropic SDK
- OpenAI Agents SDK
- LangGraph
- CrewAI
- Smolagents (Hugging Face)

HOSTED / PLATFORM

- Amazon Bedrock Agents
- Google Vertex AI Agent Builder
- Semantic Kernel (Microsoft)

TOOLBOX

LLM APIs

Anthropic API
Claude models

OpenAI API
GPT / o-series

Google Vertex AI
Gemini

Together / Groq
Open-source hosting

Agent Frameworks

LangGraph
Stateful graph workflows

CrewAI
Role-based agent teams

AutoGen
Multi-agent chat (MSFT)

Smolagents
Lightweight (HuggingFace)

Vector Databases

ChromaDB
Simple, local-first

Pinecone
Managed, enterprise

Weaviate
Multi-modal search

pgvector
Postgres extension

Protocols

MCP
Agent ↔ tool standard

A2A
Agent ↔ agent protocol

JSON-RPC
Underlying transport

REST / SSE
Streaming interfaces

Dev & Coding

Claude Code
Agentic CLI coding

GitHub Copilot
Inline code assist

Cursor
AI-native IDE

Replit Agent
Full-stack app builder

Observability & Eval

LangSmith
Tracing & evals

Weights & Biases
Experiment tracking

Braintrust
LLM eval platform

Phoenix (Arize)
Open-source traces

Data & Research

FRED API
Fed economic data

Pandas / Polars
Data manipulation

Pydantic
Typed structured output

Crawl4AI
LLM-ready web scraping

§2 YOUR FIRST API CALL

```
import anthropic

client = anthropic.Anthropic()  # reads API key from env

response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=512,
    system="You are a quant economics RA...",
    messages=[{"role": "user",
               "content": "Explain the yield curve."}]
)
```

Model

claude-sonnet-4-20250514 — fast, capable, cost-effective

System Prompt

Shapes behavior: "Be precise, cite data, don't hedge"

Response

response.content[0].text + usage stats (tokens, cost)

§2 STRUCTURED OUTPUTS WITH PYDANTIC

Define the Schema

```
class MacroIndicator(BaseModel):
    name: str
    fred_series_id: str
    latest_value: float
    unit: str
    direction: str
    significance: str

class MacroSnapshot(BaseModel):
    indicators: list[MacroIndicator]
    overall_assessment: str
```

Why Structured Outputs?

- Typed, validated data — not free-form text
- Every field has a known type (str, float, etc.)
- Pydantic catches malformed responses automatically
- Enables programmatic downstream use
- Foundation for agent-to-agent communication

Warning: LLM-generated numbers look authoritative but may be outdated training-data values.

§3 DEFINING THE FRED TOOL

```
tools = [{
  "name": "get_fred_series",
  "description": "Fetch economic
    time series from FRED...",
  "input_schema": {
    "type": "object",
    "properties": {
      "series_id": {
        "type": "string",
        "description": "FRED ID"
      },
      "start_date": {...},
      "n_recent": {...}
    },
    "required": ["series_id"]
  }
}]
```

Tool Design Tips

- Write descriptions as if explaining to a smart colleague
- Include common series IDs in the description
- Only require truly mandatory parameters
- Return JSON strings — Claude parses them
- Always test tools independently before wiring into agent

§4 THE ReAct AGENT LOOP

Pseudocode

```
messages = [user_query]

for iteration in range(max_iter):
    response = claude(messages, tools)

    if response.stop_reason == "end_turn":
        return response.text  # Done!

    if response.stop_reason == "tool_use":
        for tool_call in response:
            result = execute(tool_call)
            messages.append(result)
        # Loop continues...
```

Key Signals

`stop_reason='end_turn'`
→ Claude is done, return answer

`stop_reason='tool_use'`
→ Execute tools, loop again

Safety Features

- max_iterations prevents infinite loops
- Token tracking for cost control
- Tool registry validates function names
- Error handling in tool execution

§4 AGENT IN ACTION — MULTI-TOOL REASONING

"Assess U.S. recession risk using the yield curve, unemployment, and consumer sentiment."

Iter 1

Calls get_fred_series(T10Y2Y), get_fred_series(UNRATE), get_fred_series(UMCSENT)

Iter 2

Synthesizes all three data sources → delivers grounded recession risk assessment

Agent Output Highlights

Yield Curve (T10Y2Y)	`+0.50% → Normal curve, LOW risk`
Unemployment (UNRATE)	`4.3% → Elevated but stable, MIXED`
Consumer Sentiment	`56.6 → Severely depressed, HIGH risk`

KEY TAKEAWAYS

API Call	client.messages.create() — model, max_tokens, messages
System Prompt	Persona + rules = shaped, consistent behavior
Structured Outputs	Pydantic + JSON schema → typed, validated data
Tool Calling	Define functions as JSON, Claude decides when to use them
Tool Execution	stop_reason='tool_use' triggers your code
ReAct Loop	while loop until 'end_turn' — autonomous multi-step reasoning